Process and Decision Documentation

Project/Assignment Decisions

Player Stats Rework

- Upon getting the unedited code from Claude, I decided to completely rework the stamina and echolocation cooldowns and timers.
- The base stamina was too high which made the game far too easy. The echolocation timer also allowed the ability to be used too often which made the game overly simple as well.
- Michael Gvozdek was responsible for these changes.

Echolocation and Vision Refinement

- The echolocation mechanic was functional from the AI's original output, but it needed refinement to preopoerly convey my artistic vision for the game.
- The map was very bright originally which didn't give much reason for the mechanic to be used. By darkening the map and reducing the players field of vision/fog of war, the echolocation mechanic became more useful and necessary.
- Michael Gvozdek was responsible for these changes.

Control Layout and Tutorial Updating

- Upon our first playtests we realized that some of the controls were redundant and that the tutorial had too much information. I removed the space bar as a jump/fly button and updated the tutorial to remove unnecessary information.
- Having too many buttons for the player to control the character with would create a **cognitive overload** for the player. We noticed during our playtests that players already had a hard time remembering certain buttons like 'E' for echoloaction, so removing the space bar as an alternative to jump/fly reduced the cognitive load on the player.
- Michael Gvozdek was responsible for these changes.

Texture Creation and Planning (Textures not added yet)

- The look for the bat, platforms, fruit and spikes are all placeholders from what the AI gave us. We came up with a plan to allow us to easily replace these textures with custom ones made by our team. We ensured that the platform textures were tileable and that they would align with the models and hitboxes that were currently in the game.

- Thinking about using custom textures and sprites early on allowed us to plan ahead and avoid troubles that would slow us down in the future.
- Michael Gvozdek and Charlie Cilla Coordinated graphic asset planning.

Audio System Implementation

- The base game from Claude had no audio whatsoever. A complete audio system was built from scratch using the browser's Web Audio API, integrating seven sound assets across two categories: looping ambient sounds and one-shot sound effects.
- The looping sounds (wing flaps, footsteps, and background music) were designed to respond directly to the player's state — wings play when airborne, footsteps play when moving on the ground, and the BGM fades in and out at the right moments. The one-shot effects (apple crunch, damage hit, sonar ping, and tutorial whoosh) were each tied to specific game events.
- Several bugs were caught and fixed during testing: the damage sound was triggering every frame during the death animation causing a loud looping screech, the walk sound was bleeding into level transition screens, and the BGM was failing to restart after death due to a timing conflict with the fade-out. An AudioContext suspension issue causing a delayed whoosh on tutorial page turns was also diagnosed and resolved.
- All audio files were sourced from Mixkit and Pixabay. Audio implementation and integration was handled by Jowan Manjooran Jomon.

GenAI Documentation

1) Date/s Used: Feb 28, Mar 2, Mar 11

Tool Disclosure: tool name + model/version: Claude: Sonnet 4.6

Purpose of Use: To generate a plan for the initial code structure and file organization, as well as generate the majority of the base code for the game.

Summary of Interaction: Initially, Claude was used to figure out how we could organize our files properly, as well as what files we would need to meet the requirements of our game. Before asking it to code anything, we gave the AI a very detailed prompt including the requirements and features for our game, as well as what we had planned for it in the future. This lets it generate a plan and structure of exactly what we would need to be successful.

Once the structure and plan were set up, Claude was asked to generate the base code. The code was then formatted and brought into VSCode. After human review, several changes were made manually, and Claude was prompted again to update certain mechanics and features.

Human Decision Point(s): The gameplay was too simple from the straight AI code, so variables like total stamina, stamina drain, the echolocation timer, and more were changed to make the game more challenging and immersive. The echolocation mechanic was updated manually to enhance its visual effects. Text for the tutorial was updated to better portray our mechanics and controls. A fully custom level was also added to test the level creation implementation for the future. The controls were updated to remove redundant and broken buttons.

Integrity & Verification Note: The AI's action plan and file structure were analyzed to ensure they aligned with assignment requirements as well as to ensure that they supported the files we had planned for the future.
Once the AI had given us code, all of the code was checked over to ensure it was properly formatted and efficient. We specifically checked to see if the AI had used variables for aspects like colour and player stats, because if it hadn't, changing that info would be a problem in the future.

Scope of GenAI Use:
We used GenAI to generate the structure and base code for our gameplay. No GenAI was used to come up with the concept or ideas of our game, along with its main features and mechanics.

Limitations or Misfires:
In order to make the creation of new levels easier for us to visualize, we prompted Claude to create a spreadsheet that replicated the row and column structure of the levels.js data file. This sheet used colour coordination to make the level easy to visualize rather than just looking at numbers. A problem that occurred with this is that it was hard to change the size of the map in the sheet, and when we prompted the AI to change it for us, it outputted the same thing we put in. We ended up sticking with the

same map size for the new level, but we will have to figure out a solution in the future for larger maps.

Summary of Process (Human + Tool)

Describe what you did, focusing on process rather than outcome.

- Created a prompt which described the concept of our gameplay, including the main features and mechanics we wanted to incorporate. In the initial prompt, we asked the platform to provide the file structure to create the game.
- Claude responded with a clean file structure and a few notes describing the functions.
- After analyzing the file structure provided by Claude, we asked it to provide the code for our game.
- Claude responded by refining the file structure and cleaning the data using p5j.s
- Using the file and folder structure, along with the overall code provided by Claude, we pushed the game to GitHub and tested out the initial gameplay.
- While playtesting the game, we came across a few changes we wanted to make with the visuals. Therefore, we prompted Claude to assist with these changes and improve our gameplay.

Decision Points & Trade-offs

Describe one or two key decisions you made:

- Decision #1: We came across an issue when we prompted Claude to create a spreadsheet that replicated the row and column structure of the levels.js data file. We discovered that it was too difficult to change the size of the map within the sheet, and decided to troubleshoot this problem during the next phase of the project.

Verification & Judgement:

Using the file structure given by Claude, we referred to the VS Code format and organization to verify that the information was stored correctly. We referenced the p5.js editor to make edits to the code, such as the 'levels' file. As a group, we individually analyzed and read over the code to make sure it made sense and was accurate to our ideas.

Limitations, Dead Ends, or Open Questions

One open question for the next phase of the project is how we can adjust the size of the map within the sheet.

Appendix

https://claude.ai/share/51266161-bf57-4570-a3b5-d0ca03d9ddc5

2) Date/s Used: Mar 10–11, 2026

Tool Disclosure: tool name + model/version: Claude Code (CLI): Claude Sonnet 4.6

Purpose of Use: To design, implement, debug, and polish a complete audio system for the game.

Summary of Interaction: Claude was used to build a self-contained AudioManager.js from scratch using the Web Audio API. Before writing any code, I explored the existing project structure to understand how the game loop, player state, and input handling worked. Audio was added, starting with wing flap and footstep loops, then one-shot SFX (apple crunch, damage, sonar ping, tutorial whoosh), and finally looping background music. Each addition was reviewed before moving to the next.

Human Decision Point(s): All audio files were sourced and named by the team. Decisions about which game events should trigger sounds (echolocation, fruit collection, damage, walking vs. airborne) were made by the team. Volume levels and fade timing were tuned through playtesting. The team identified a looping death-scream bug and a delayed whoosh bug, which were then fixed. The team also decided to reorganize audio files into assets/audio/.

Integrity & Verification Note: All generated code was reviewed before committing. The team caught the death-sound loop bug (|| player.dead firing 60 times), verified the BGM restart logic after level transitions, and confirmed the AudioContext suspension fix addressed the timing delay.

Scope of GenAI Use:
GenAI was used to write and debug the audio implementation only. Sound design, asset selection, and all gameplay decisions were made by the team.

Limitations or Misfires:
The first attempt at BGM timing failed because fadeIn() silently bailed when the audio buffer hadn't decoded yet. A _bgmPending flag was added to handle this async gap. The initial hurt sound also triggered every frame during the death animation, which was caught and fixed during review.

Summary of Process (Human + Tool)

- All audio assets (wings, footsteps, apple crunch, damage, sonar, whoosh, BGM) were sourced and added to the project by the team. Claude was then asked to wire them into the game through a new, isolated AudioManager.js file.
- Before writing any code, Claude read the existing project files to understand the game loop, player state flags, and input handling in main.js and Player.js.
- Audio was added one by one across multiple prompts, starting with the wing flap loop, then walking, then one-shot SFX, then background music. Each piece was tested by the team before the next was added.
- After playtesting, the team identified two errors/bugs: the damage sound looping on' death', and the walking sound continuing into level transition screens. These were reported to Claude.
- The team also identified that the tutorial whoosh was delayed, which Claude traced to the browser suspending the AudioContext and fixed with a resume() call.
- Audio files were reorganized into assets/audio/ by the team, and Claude updated the file paths accordingly.

Decision Points & Trade-offs

- Decision #1: The team decided to keep all audio logic in a single separate file (AudioManager.js) rather than embedding it in main.js or Player.js. This made it easy to review, adjust volumes in one place, and revert entirely if needed.
- Decision #2: The team chose to keep the BGM continuous across levels rather than restarting it each time, so transitions feel smoother. When a level ends, the BGM fades out and fades back in when the next level loads.

Verification & Judgement:

Each sound was tested in-game after being added. The team caught both the death-loop bug and the level-transition bleed by playtesting the game. Volume levels were reviewed as a mix and checked that SFX (crunch, hurt) clearly cut through the ambient loops (wings, walk) and BGM.

Limitations, Dead Ends, or Open Questions:

The BGM timing on first load depends on how fast the browser decodes the audio file. A _bgmPending flag was added to handle this, but on very slow connections, there may still be a brief silence before the BGM begins.

Appendix

*** Claude Code (VSCode extension) does not generate shareable conversation links. Screenshot of the session are attached below with the whole transcript

Thinking

The user says "ok could we fix" but doesn't specify what to fix. They might be referring to something from the previous conversation, or they might have cut off their message. Let me ask them to clarify what they want to fix.

It looks like your message got cut off — what would you like to fix?

i attahced. awav.game bgm for level music its a bit uppity make sure you blend well usimg other sounds

Thinking

The user attached a WAV file for background music (BGM) for level music. Let me check what file was added to the assets folder.

Bash List assets

```
IN  ls "/Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/(
OUT AppleCrunch.wav
    damageHurt.wav
    gameBgm.mp3
```

Thinking

The BGM file is `gameBgm.mp3`. Now I need to:

1. Add it to AudioManager as a looping background track
2. Play it when the game starts playing (when entering PLAYING state)
3. Stop it when leaving PLAYING state
4. Balance its volume against the other sounds

Since the user says it's "a bit uppity" and wants it to "blend well with other sounds", I should:

- Set BGM at a lower volume (maybe 0.25-0.30) so it sits underneath

Edit AudioManager.js

Added 1 line

```
// ── Volume levels (tune here) ──────────────────────────
const VOL_WINGS = 0.55;   // airborne wing flaps
const VOL_WALK  = 0.38;   // ground footsteps (slightly
quieter than wings)
const VOL_WINGS  = 0.55;  // airborne wing flaps
const VOL_WALK   = 0.38;  // ground footsteps (slightly
quieter than wings)
const VOL_CRUNCH = 0.70;  // apple collect — punchy
one-shot feedback
```

Edit AudioManager.js

Added 2 lines

```
  constructor() {
    this._ctx     = null;
    this._wings   = null;  // _AudioTrack, set after unlock
()
    this._walk    = null;  // _AudioTrack, set after unlock
()
    this._pending = {};    // raw ArrayBuffers fetched
before ctx exists
    this._ctx       = null;
    this._wings     = null;  // _AudioTrack, set after
    unlock()
```

Edit AudioManager.js

Added 19 lines

```
    this._ctx   = new (window.AudioContext || window.
    webkitAudioContext)();
    this._wings = new _AudioTrack(this._ctx, VOL_WINGS);
    this._walk  = new _AudioTrack(this._ctx, VOL_WALK);
    // Decode any audio that already finished fetching.
    for (const key of ['wings', 'walk']) this._tryDecode
(key);
    for (const key of ['wings', 'walk', 'crunch']) this.
```

i added sssets and an audio file names w...

Minimal changes to existing files:

- index.html — added one `<script>` tag for AudioManager
- src/main.js — 4 small additions:
  1. `let audioManager;` global
  2. `audioManager = new AudioManager()` in setup
  3. `audioManager.unlock()` on first keypress (required for browser autoplay policy)
  4. `audioManager.update(player)` each frame in `drawPlaying` — plays the loop when bat is airborne (`!isHanging`), stops when it lands or dies

To revert: delete `AudioManager.js`, remove its `<script>` tag from `index.html`, and remove the 4 `audioManager` lines from `main.js`.

can we ease the audio when its cut off

index.html

Thinking

The user wants to fade out the audio instead of abruptly cutting it off when the bat lands or dies. I need to update the AudioManager.js to use a gain ramp for smooth fade-out instead of immediately stopping.

Read AudioManager.js

Write AudioManager.js

112 lines

```
// ── AudioManager.js ──────────────────────────────────
// Handles wings flap sound. Completely self-contained — remove thi
// and its <script> tag + the three audioManager calls in main.js t
```

The audio now fades smoothly in both directions:

- **Lift-off** — 0.12s fade in when the bat leaves a surface
- **Landing / dying** — 0.35s fade out instead of an abrupt cut

i added sssets and an audio file names w...

Let me explore the project structure first.

Agent: Explore game project structure

```
IN  Explore the project at /Users/ncs/Desktop/UW/3B/GBDA302 Global
    1. All JS files (especially main game files, any existing audi
    2. All asset files (audio files especially anything named "wir
```

Bash

```
IN  find "/Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project
OUT /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
    /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
    /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
```

Bash

```
IN  ls -laR "/Users/ncs/Desktop/UW/3B/GBDA302 Global Digital P
OUT total 48
    drwxr-xr-x@ 13 ncs  staff   416 Mar 11 19:04 .
    drwxr-xr-x  18 ncs  staff   576 Mar  8 13:36 ..
```

Bash

```
IN  find "/Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project
OUT /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
    /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
    /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
```

Bash

```
IN  find "/Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project
OUT /Users/ncs/Desktop/UW/3B/GBDA302 Global Digital Project/Grou
```

Appendix

https://docs.google.com/document/d/1zSS35QbNB65SKGkXdbUpM2pKXCkxMiF0EbjO0j-DKsY/edit?usp=sharing

References

Mixkit. (n.d.). *Bat wing flap loop* [Sound effect]. Mixkit. https://mixkit.co

Mixkit. (n.d.). *Pavement walk footstep loop* [Sound effect]. Mixkit. https://mixkit.co

Mixkit. (n.d.). *Apple crunch* [Sound effect]. Mixkit. https://mixkit.co

Mixkit. (n.d.). *Damage hurt hit* [Sound effect]. Mixkit. https://mixkit.co

Mixkit. (n.d.). *Whoosh transition* [Sound effect]. Mixkit. https://mixkit.co

Pixabay. (n.d.). *Sonar ping* [Sound effect]. Pixabay. https://pixabay.com

Pixabay. (n.d.). *Game background music* [Music]. Pixabay. https://pixabay.com